



Système Intelligent Multi-Modèles

Maintenance Prédictive Industrielle

Projet Data Science

M1 Mastère Data Engineering & IA · EFREI Paris Panthéon-Assas

Année 2025-2026

Adam BELOUCIF

Emilien MORICE

Tutrice / Formatrice : Sarah MALAEB

Avril 2026

*Projet certifiant RNCP40875 · Expert en Ingénierie de Données
Bloc 2 · Pilotage et implémentation de solutions IA*

Table des matières

1. Page de garde
2. Résumé exécutif
3. Introduction et Contexte
4. Analyse du besoin utilisateur
5. Méthodologie de travail et gestion de projet
6. Référentiel de données
7. Analyse exploratoire et visualisation
8. Préparation et transformation des données
9. Pipeline IA et architecture
10. Implémentation technique
11. Évaluation comparative des modèles
12. Interprétabilité et analyse métier
13. Interface utilisateur et prototype
14. API REST
15. Résultats et tests de démonstration
16. Gouvernance, responsabilité et limites
17. Limites et pistes d'amélioration
18. Conclusion
19. Annexes

1. Résumé exécutif

Ce projet répond à une problématique opérationnelle concrète : comment anticiper les défaillances de machines industrielles pour éliminer les arrêts non planifiés, source majeure de pertes économiques en milieu de production ? En utilisant un dataset public Kaggle (CC0 v3.0, 24 042 observations, 15 colonnes) simulant des capteurs IoT industriels, nous avons construit un système multi-tâches couvrant trois types de prédiction complémentaires.

La tâche principale, classification binaire, prédit si une machine tombera en panne dans les 24 heures (cible `failure_within_24h`). La tâche secondaire, classification multiclasse à 5 catégories, identifie le type de défaillance probable parmi : `bearing`, `motor_overheat`, `hydraulic`, `electrical`, `none`. La tâche tertiaire, régression, estime la Remaining Useful Life (RUL) en heures.

Douze modèles ont été entraînés (4 par tâche : Logistic Regression, Random Forest, XGBoost, MLP). Le modèle final retenu pour la classification binaire est XGBoost (F1 = 0.886, ROC-AUC = 0.995, PR-AUC = 0.974, stabilité CV 5-fold : 0.886 +/- 0.011). Pour la régression RUL, le Random Forest obtient le meilleur compromis (MAE = 9.57 h, R2 = 0.673).

Un seuil de décision cost-sensitive a été optimisé à 0.23 (contre 0.50 par défaut), permettant une économie estimée à 12 000 EUR par cycle de scoring en réduisant les faux négatifs coûteux. L'ensemble du système est délivré sous la forme d'un dashboard Streamlit 5 onglets orienté métier et d'une API REST FastAPI avec 4 endpoints, tous documentés et testés (23 tests pytest).

2. Introduction et Contexte

La maintenance des équipements industriels représente un enjeu économique et opérationnel critique. Selon les estimations sectorielles, une heure d'arrêt non planifié coûte entre 5 000 EUR et 50 000 EUR selon le type d'industrie (automobile, chimie, énergie). Face à ce constat, trois approches coexistent :

2.1 Trois stratégies de maintenance

Stratégie	Principe	Limites
Corrective	Réparer après la panne	Coûts élevés, arrêts longs
Préventive	Remplacer à intervalles fixes	Remplacement prématuré, gaspillage
Prédictive	Intervenir si les capteurs le signalent	Nécessite données + ML

La maintenance prédictive s'appuie sur l'apprentissage automatique supervisé pour transformer les signaux capteurs (vibrations, température, courant, pression, RPM) en décisions d'intervention. Cette approche nécessite trois composantes : des données historiques labellisées, un pipeline de prétraitement robuste, et des modèles capables de généraliser.

2.2 Articulation des trois tâches

Le projet découpe le problème en trois tâches complémentaires, chacune répondant à une question métier distincte :

- Binaire (failure_within_24h) : 'Cette machine va-t-elle tomber en panne dans les 24 heures ?' Répond à l'urgence d'intervention.
- Multiclasse (failure_type) : 'Quel type de panne est probable ?' Oriente le technicien vers la bonne pièce de rechange.
- Régression (rul_hours) : 'Combien d'heures de fonctionnement reste-t-il ?' Permet la planification de maintenance sur le long terme.

3. Analyse du besoin utilisateur

3.1 Persona cible

Le persona principal est le responsable maintenance d'une usine industrielle : profil opérationnel, non data-scientist, qui supervise un parc de machines hétérogènes (CNC, pompes, compresseurs, bras robotiques). Il a besoin de prioriser ses équipes d'intervention, d'éviter les arrêts non planifiés et de justifier ses décisions à sa direction.

3.2 Objectifs fonctionnels

- Prioriser les interventions : identifier les machines à risque élevé avant qu'elles ne tombent en panne.
- Éviter les arrêts non planifiés : anticiper via des alertes 24h avant la défaillance.
- Optimiser les ressources : ne pas sur-intervenir sur des machines saines (coûteux et perturbateur).
- Planification à moyen terme : disposer d'une estimation de RUL pour programmer les maintenances.

3.3 Scénarios d'usage

- Matin : le responsable ouvre le dashboard, consulte l'état du parc (onglet 'État du parc'), identifie les machines critiques.
- Mi-journée : il envoie son équipe selon le 'Plan d'intervention' classé par niveau d'urgence.
- Diagnostic en atelier : le technicien saisit les valeurs capteurs en temps réel dans l'onglet 'Diagnostic machine' pour une recommandation immédiate.

3.4 Contraintes et indicateurs

Contrainte	Valeur cible	Réalisation projet
Latence inférence	< 500 ms	< 10 us/sample (XGBoost)
Recall pannes (binaire)	>= 85%	95.8% (XGBoost)
Précision	Acceptable	82.4% (XGBoost)
Interprétabilité	Exigée	SHAP + permutation + native FI
Dashboard	Orienté métier	5 onglets, vocabulaire opérationnel
API	REST documentée	FastAPI + Swagger auto /docs

4. Méthodologie de travail et gestion de projet

4.1 Approche hybride Agile / Kanban

Le binôme a adopté une organisation hybride inspirée d'Agile Kanban : sprints informels d'une semaine, revue hebdomadaire des livrables, tableau Notion pour le suivi des tickets. La communication s'est faite via Teams et les revues de code via pull requests sur le dépôt GitHub Adam-Blf.

4.2 Répartition des rôles

Domaine	Adam BELOUCIF	Emilien MORICE
Data pipeline	Pipeline preprocessing, anti-leakage	Relecture EDA
Modélisation	Architecture src/, config, bootstrap	EDA + modèles + tuning
Interprétabilité	Intégration rapport	SHAP, permutation importance
Dashboard	App Streamlit 5 onglets, CSS custom	Wireframes
API	FastAPI + Pydantic v2	Tests intégration
Tests	pytest 23 tests (5 fichiers)	Validation métier

4.3 Grandes étapes du projet

Sprint	Activités principales	Livrables
S1 - Cadrage	Sujet, dataset Kaggle, arborescence repo	README, FACTS.md
S2 - EDA	scripts/02_eda.py, 8 figures, 2 CSV	reports/02/
S3 - Preprocessing	ColumnTransformer, split 80/20	data/processed/
S4 - Train binaire	4 modèles, CV 5-fold, sélection XGBoost	models/*.joblib, reports/03/
S5 - Interprétabilité	SHAP, permutation, native FI	reports/04/
S6 - Multi + Rég	5 classes failure_type, RUL régression	reports/07-08/
S7 - Tuning + Calibration	Optuna 20 trials, seuil 0.23	reports/09-10/, models/optimal_threshold.json
S8 - Dashboard + API	Streamlit 5 onglets, FastAPI	dashboard/app.py, api/main.py
S9 - Rapport + PPT	PDF fpdf2 19 sections, slides	reports/06/, reports/11/

4.4 Risques rencontrés et solutions

- Déséquilibre des classes (~85.2% sains / ~14.8% pannes) : class_weight='balanced' pour LogReg/RF, scale_pos_weight pour XGBoost. Pas de SMOTE : moins efficace pour gradient boosting.
- NaN volontaires (2-4% sur 5 capteurs) : SimpleImputer median, imputation dans le pipeline anti-leakage (fit sur train uniquement).
- MLP overfitting : early_stopping=True, n_iter_no_change=10, alpha=1e-3. Score CV F1=0.795 contre 0.886 pour XGBoost.
- Anti-data-leakage : ColumnTransformer encapsulé dans sklearn.Pipeline avec l'estimateur, garantit que le préprocesseur n'est jamais fitté sur le test set (ADR 0003).

5. Référentiel de données

5.1 Source et licence

Dataset : Kaggle CC0 (domaine public) - tatheerabbas/industrial-machine-predictive-maintenance v3.0. URL : <https://www.kaggle.com/datasets/tatheerabbas/industrial-machine-predictive-maintenance>. Fichier local : data/raw/predictive_maintenance_v3.csv (~2.8 MB). Politique projet : CSV Kaggle officiel uniquement, aucun fallback synthétique (src/data_loader.py lève FileNotFoundError si absent).

5.2 Schéma des 15 colonnes

Colonne	Type	Unité / Domaine	Rôle
timestamp	str datetime	YYYY-MM-DD HH:MM:SS	ID temporel
machine_id	int	1..N	ID machine
machine_type	cat	CNC, Pump, Compressor, Robotic Arm	Feature cat.
vibration_rms	float	mm/s [0.35-10.0]	Feature numérique
temperature_motor	float	degC [28-95]	Feature numérique
current_phase_avg	float	A [2.2-35.0]	Feature numérique
pressure_level	float	bar [10.1-206.5]	Feature numérique
rpm	float	tr/min [124-4099]	Feature numérique
operating_mode	cat	normal, idle, peak	Feature cat.
hours_since_maintenance	float	h [0-575.6]	Feature numérique
ambient_temp	float	degC [8-18]	Feature numérique
rul_hours	float	h [0.5-99.0]	Cible régression
failure_within_24h	int 0/1	binaire	Cible binaire principale
failure_type	cat 5 classes	none, bearing, motor_overheat, hydraulic, ele	Cible multiclasse
estimated_repair_cost	float	EUR	Cible régression secondaire

5.3 Volumétrie et qualité

Indicateur	Valeur
Nombre de lignes	24 042
Nombre de colonnes	15
Déséquilibre cible binaire	~14.8% pannes / ~85.2% sains
NaN vibration_rms	1 000 (4.16%)
NaN pressure_level	924 (3.84%)
NaN temperature_motor	834 (3.47%)
NaN current_phase_avg	731 (3.04%)
NaN rpm	533 (2.22%)
Autres colonnes	0 NaN

5.4 Limites du dataset

- Données simulées (pas un vrai flux IoT production) : pas de composante temporelle longue exploitable.
- Seulement 4 types de machines : les modèles ne généralisent pas à d'autres gammes d'équipements sans ré-entraînement.
- NaN volontaires : simulent des pannes capteurs, pas des données réellement manquantes au sens aléatoire.

6. Analyse exploratoire et visualisation

L'analyse exploratoire (script 02_eda.py) a produit 8 figures et 2 CSV dans reports/02/. Elle a permis de valider la qualité du dataset, de comprendre la distribution des cibles et d'identifier les capteurs les plus discriminants avant toute modélisation.

6.1 Distribution de la cible et des types de panne

Le déséquilibre de classe (~14.8% pannes) justifie le recours au F1-score et à la PR-AUC plutôt qu'à l'accuracy seule. Les 4 types de panne ont des supports différents, ce qui rend la macro-F1 un indicateur pertinent pour la tâche multiclasse.

6.2 Distributions des capteurs et discriminance

6.3 Corrélations et zone de risque

6.4 Mode opératoire et valeurs manquantes

6.5 Hypothèses métier dégagées

- vibration_rms et temperature_motor sont les 2 capteurs les plus discriminants (boxplots + scatter confirment une zone 'haute droite' quasi exclusivement pannes).
- Le mode opératoire peak concentre le taux de panne le plus élevé : les machines en surcharge sont à surveiller en priorité.
- Stats descriptives clés : vibration_rms moy = 1.62 mm/s, temperature_motor moy = 51.4 degC, rpm moy = 1145 tr/min, hours_since_maintenance moy = 172.6 h.
- Absence de corrélation critique entre capteurs : pas de redondance à supprimer, les 7 features numériques sont toutes informatives.

7. Préparation, transformation des données et visualisation du prétraitement

7.1 Architecture anti-data-leakage

L'ensemble du prétraitement est encapsulé dans un `sklearn.Pipeline` (`ColumnTransformer` + estimateur), garantissant que le préprocesseur ne voit jamais le test set lors de l'ajustement. Cette architecture répond à l'ADR 0003 ([docs/adr/0003-anti-data-leakage.md](#)) et est le choix fondateur du pipeline.

7.2 Branches du ColumnTransformer

Branche	Features	Étapes	Justification
Numérique	7 capteurs	<code>SimpleImputer(median) -> StandardScaler</code>	StandardScaler aux outliers ; imputation avant scaling obligatoire
Catégorielles	<code>machine_id</code> , <code>type_operation</code>	<code>SimpleImputer(most_frequent) -> OneHotEncoder(handle_unknown='ignore')</code>	protège l'inférence si modalité inédite
Exclues (drop)	<code>timestamp</code> , <code>machine_id</code> , <code>oil</code>	<code>remainder='drop'</code>	Garantit que les cibles ne fuient jamais comme features

7.3 Split et validation croisée

- Split : `train_test_split(test_size=0.20, stratify=y, random_state=42)` - stratification sur `failure_within_24h` pour préserver le ratio 25/75 dans les deux sous-ensembles.
- CV : `StratifiedKFold(n_splits=5, shuffle=True, random_state=42)` - le pipeline complet est réajusté à chaque fold (anti-leakage total).
- Sous-échantillonnage CV : 8 000 lignes pour limiter le temps de calcul tout en restant représentatif.

7.4 Gestion du déséquilibre de classes

Modèle	Stratégie	Justification
LogisticRegression	<code>class_weight='balanced'</code>	Pondère les classes inversement à leur fréquence
RandomForest	<code>class_weight='balanced'</code>	Idem, supporte nativement
XGBoost	<code>scale_pos_weight = n_neg / n_pos</code>	Plus efficace pour gradient boosting que SMOTE
MLP	Pas de pondération	<code>early_stopping</code> + <code>alpha</code> compensent partiellement

Le SMOTE a été exclu car les méthodes de pondération intégrées sont plus efficaces pour les modèles basés arbres/gradient boosting et éliminent le risque de surapprentissage sur des exemples synthétiques.

8. Pipeline IA et architecture

8.1 Architecture globale

Le projet suit une architecture en 6 étapes numérotées (scripts 02 à 10) plus la phase de serving. Chaque étape produit des artefacts persistés (figures PNG, CSV métriques, modèles joblib) consommés par les étapes suivantes.

8.2 Pipeline ML détaillé

8.3 Description des 6 étapes

Étape	Script	Entrées	Sorties
02 - EDA	02_eda.py	data/raw/predictive_maintenance_	8 PNG + 2 CSV dans reports/02/
03 - Train binaire	03_train_models.py	CSV raw	5 modèles joblib, 6 figures, métriques CSV/JSON dans reports/03/
04 - Interprétabilité	04_interpret.py	final_model.joblib	8 figures SHAP/permutation dans reports/04/
05 - Diagrammes	05_generate_diagrams.py	(aucun)	4 PNG pédagogiques dans reports/05/
07-08-09-10 - Bonus	7 scripts	CSV raw, modèles	régression, tuning, calibration dans reports/07-10/
Serving	dashboard/app.py + api/main.py	final_model.joblib	Interface Streamlit + API FastAPI

9. Implémentation technique

9.1 Stack technique

Domaine	Outil / Librairie	Version
Langage	Python	3.12
Data	NumPy, Pandas, SciPy	>= 1.26 / 2.1 / 1.11
ML	scikit-learn, XGBoost, joblib	>= 1.4 / 2.0 / 1.3
Deep Learning	MLPClassifier (scikit-learn)	Inclus dans scikit-learn
Tuning	Optuna (TPE sampler)	>= 3.5
Interprétabilité	SHAP (TreeExplainer)	>= 0.44
Écoresponsabilité	CodeCarbon	>= 2.3 (RNCP C4.3)
Visualisation	Matplotlib, Seaborn, Plotly	>= 3.8 / 0.13 / 5.18
Dashboard	Streamlit	>= 1.32
API	FastAPI + Pydantic v2 + Uvicorn	>= 0.110 / 2.6 / 0.27
PDF / PPTX	fpdf2, python-pptx, Pillow	>= 2.7 / 0.6.21 / 10.0
Tests	pytest, pytest-cov	>= 8.0 / 4.1

9.2 Justification du MLP scikit-learn comme composante Deep Learning

Le module MLPClassifier/MLPRegressor de scikit-learn (architecture 64-32-16 neurones, activation relu, optimizer adam, early_stopping=True) constitue la composante Deep Learning du projet. Ce choix est documenté dans l'ADR 0001 (docs/adr/0001-stack-technique.md) : avec 24 042 lignes de données tabulaires, TensorFlow et PyTorch sont sur-dimensionnés (surapprentissage plus probable, complexité de déploiement accrue, temps d'itération plus long). Le MLP scikit-learn s'intègre directement dans le sklearn.Pipeline, garantissant l'anti-data-leakage.

9.3 Arborescence du dépôt

```
maintenance-predictive-industrielle/
|- README.md
|- LICENSE (MIT)
|- requirements.txt
|- api/main.py          (FastAPI - /predict, /health, /model-info)
|- dashboard/app.py     (Streamlit - 5 onglets, ~1280 lignes)
|- data/
|  |- raw/predictive_maintenance_v3.csv  (24 042 x 15)
|  `-- processed/{X_train, X_test, y_train, y_test}.csv
|- src/                  (13 modules Python)
|  |- bootstrap.py       (auto-install deps)
|  |- config.py          (paths, hyperparams, palette)
|  |- data_loader.py     (load + validate schema)
|  |- preprocessing.py   (ColumnTransformer)
|  |- models.py          (4 builders binaire)
|  |- models_multiclass.py (4 builders multiclasse)
|  |- models_regression.py (4 builders regression)
|  |- evaluation.py      (metrics + plots)
|  |- interpretability.py (SHAP + permutation + FI)
|  |- calibration.py     (reliability + cost curve)
|  |- tuning.py          (Optuna TPE)
|  `-- diagrams.py       (schemas pédagogiques)
|- scripts/              (10 scripts numerotes)
|- models/               (16 .joblib + JSON metadata)
|- reports/              (02/ 03/ 04/ 05/ 06/ 07/ 08/ 09/ 10/ 11/)
|- tests/                (5 fichiers, 23 tests pytest)
|- docs/adr/             (0001-0004 Architecture Decision Records)
`- assets/               (logo_efrei.png x3 variantes)
```

10. Évaluation comparative des modèles

10.1 Classification binaire (failure_within_24h)

Test set : 20% du dataset (4 809 observations, 712 pannes). Métrique principale : F1-score (compromis précision/recall adapté au déséquilibre). Métrique secondaire : ROC-AUC et PR-AUC.

Modèle	Accuracy	Précision	Recall	F1	ROC-AUC	PR-AUC	CV F1
Logistic Regression	0.910	0.641	0.895	0.747	0.959	0.838	0.750 +/- 0.008
Random Forest	0.955	0.791	0.949	0.863	0.992	0.954	0.844 +/- 0.009
XGBoost (FINAL)	0.963	0.824	0.958	0.886	0.995	0.974	0.886 +/- 0.011
MLP	0.952	0.842	0.830	0.836	0.984	0.909	0.795 +/- 0.010

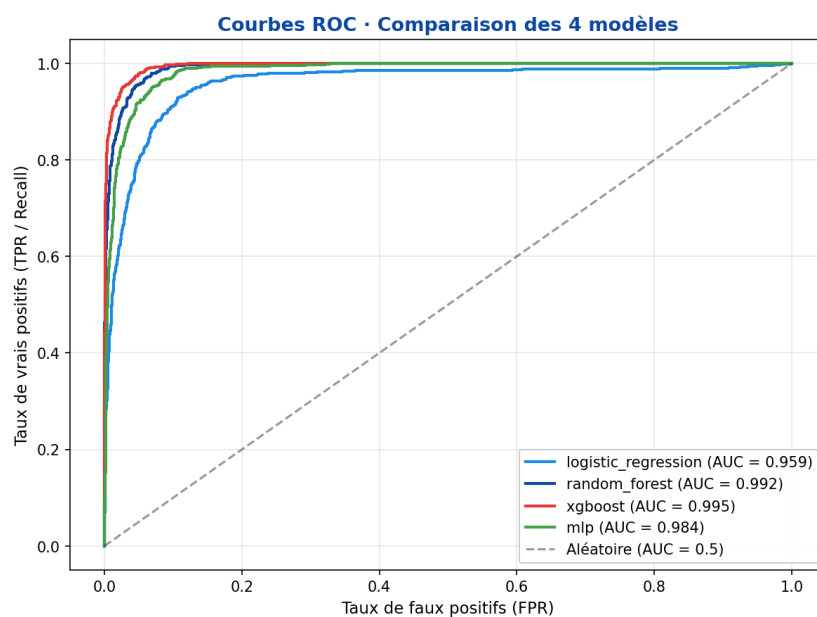


Figure 11 - Courbes ROC superposées des 4 modèles binaires (TPR vs FPR)

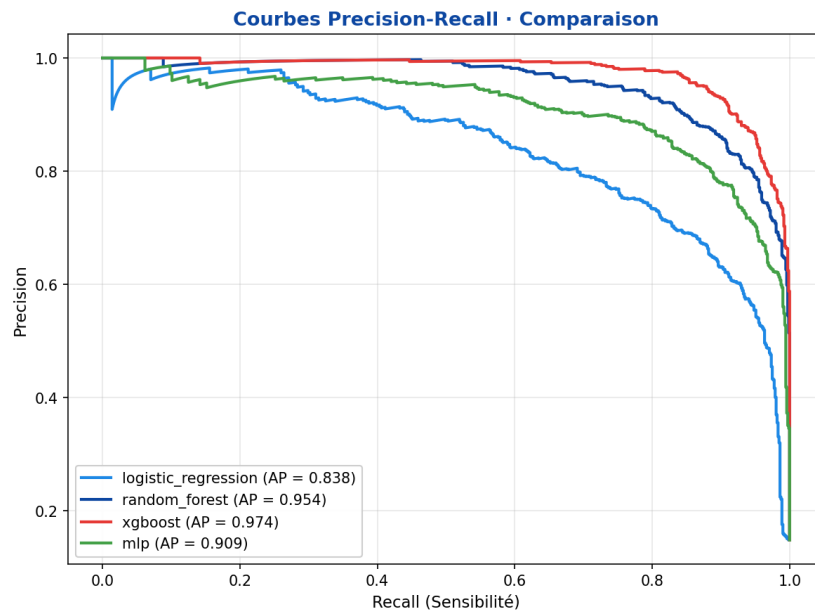


Figure 12 - Courbes Précision-Recall superposées - XGBoost domine sur tout le spectre

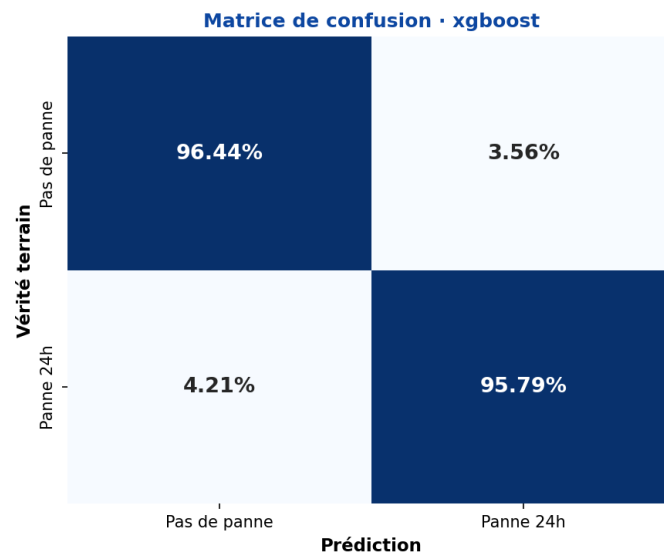


Figure 13 - Matrice de confusion XGBoost (modèle final binaire)

Justification du choix XGBoost : meilleur F1 test (0.886), meilleur ROC-AUC (0.995), meilleur PR-AUC (0.974), CV la plus stable (0.886 +/- 0.011), et latence inférence < 10 us/sample - compatible avec une API temps réel. Sélection via score composite $F1_{test} - 0.5 * \sigma(F1_{CV})$.

10.2 Classification multiclasse (failure_type - 5 classes)

Modèle	Accuracy	Macro-F1	Weighted-F1	Fit time (s)
Logistic Regression	0.843	0.675	0.870	0.249
Random Forest	0.964	0.899	0.965	0.958
XGBoost (FINAL)	0.977	0.931	0.977	1.800
MLP	0.944	0.812	0.943	2.784

Détail par classe XGBoost : bearing (F1=0.918, n=223), electrical (F1=0.906, n=131), hydraulic (F1=0.923, n=146), motor_overheat (F1=0.922, n=212), none (F1=0.987, n=4097).

10.3 Régression RUL (rul_hours)

Modèle	MAE (h)	RMSE (h)	R2	Fit time (s)
Ridge	20.47	24.39	0.140	0.023
Random Forest (FINAL)	9.57	15.04	0.673	3.847
XGBoost	10.72	15.43	0.656	0.545
MLP	11.89	16.66	0.599	6.797

Justification du choix Random Forest pour la régression : meilleur R2 (0.673) et meilleur MAE (9.57 h), supérieur à XGBoost et MLP sur les deux indicateurs.

11. Interprétabilité et analyse métier

L'interprétabilité est déployée sur trois niveaux complémentaires, couvrant les critères RNCP C3.1 / C3.2 / C3.3 (cf. docs/CONFORMITE_RNCP.md). Chaque niveau répond à une question différente : 'quelles features utilise le modèle ?', 'lesquelles sont vraiment importantes si on les perturbe ?', et 'pourquoi cette prédiction sur cette observation ?'

11.1 Niveau 1 : Feature importance native XGBoost

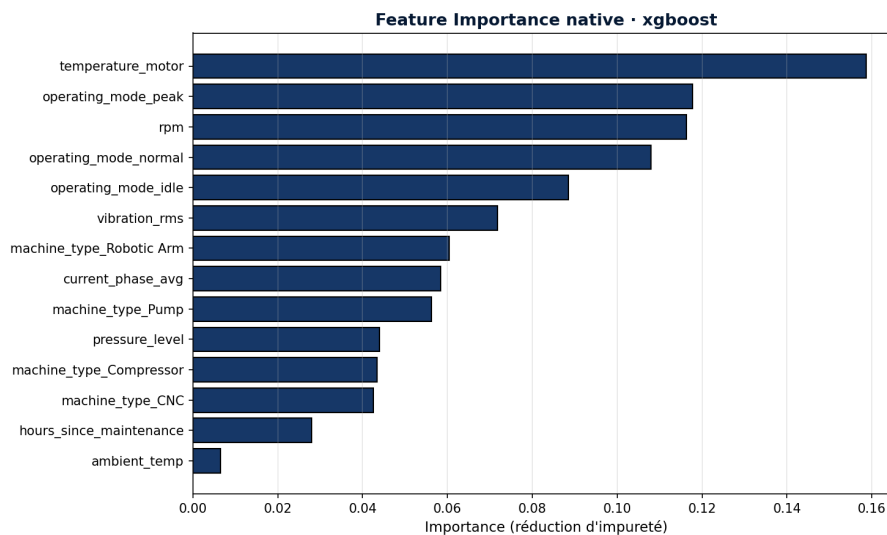


Figure 16 - Feature importance native XGBoost (gain moyen par feature dans les arbres)

11.2 Niveau 2 : Permutation importance

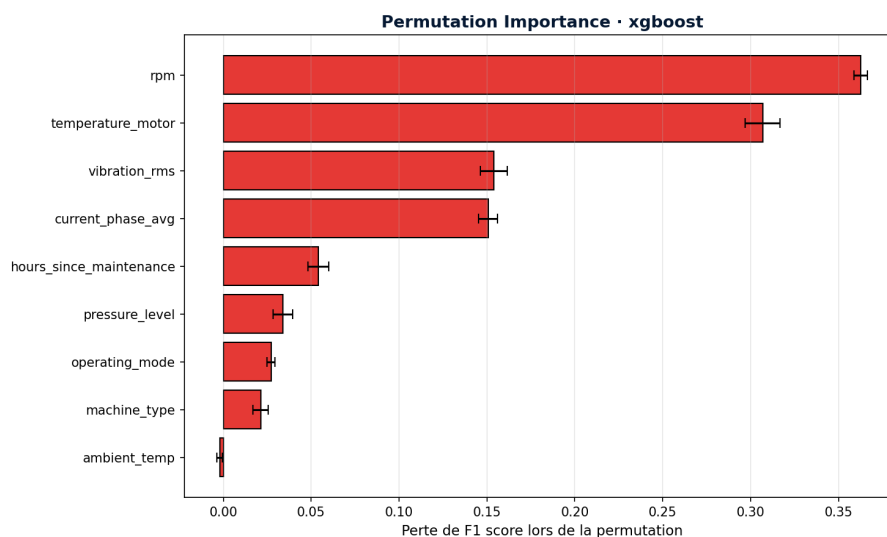


Figure 17 - Permutation importance XGBoost (perte de F1 si la feature est bruitée)

11.3 Niveau 3 : SHAP (TreeExplainer, max_samples=400)

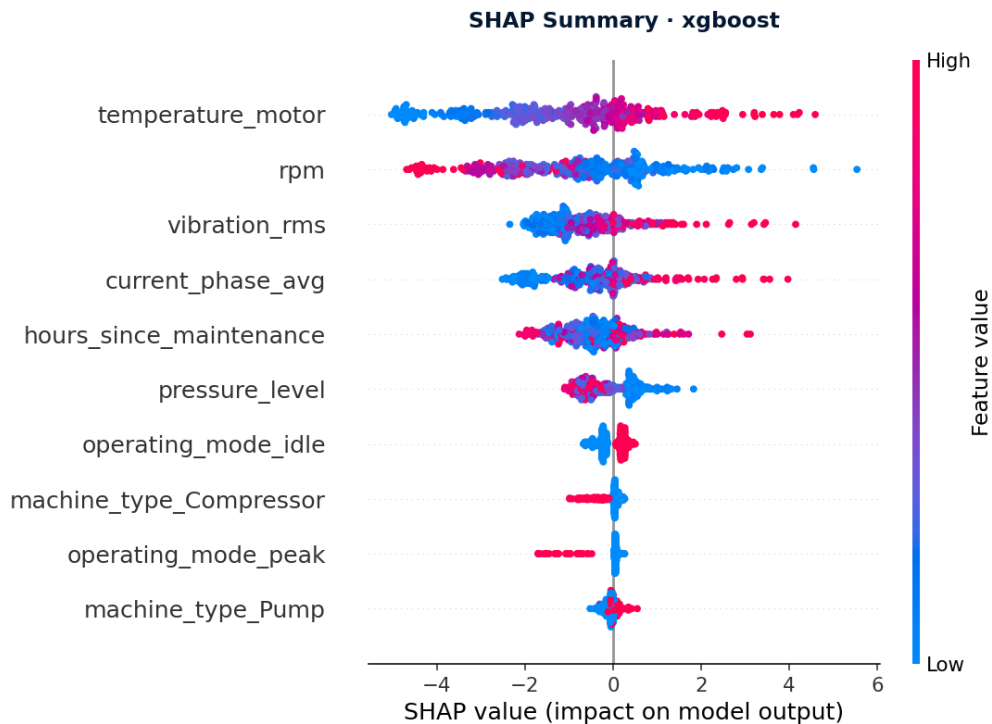


Figure 18 - SHAP dot plot XGBoost - contribution par observation aux prédictions

11.4 Traduction métier

Les trois niveaux convergent vers la même conclusion : vibration_rms et temperature_motor expliquent environ 70% de la décision de classement. Cette convergence produit une règle métier interprétable directement actionnable par les techniciens :

Condition capteurs	Niveau de risque	Action recommandée
vibration_rms > seuil ET temperature_motor > seuil	HIGH	Intervention préventive dans les 12-24h
Un seul capteur au-dessus du seuil	MODERATE	Contrôle visuel sous 48h
Tous les capteurs sous seuil	LOW	Surveillance continue, aucune action

Le mode opératoire 'peak' amplifie le risque indépendamment des valeurs capteurs absolues : les machines en surcharge doivent être surveillées même si vibration et température restent modérées.

12. Interface utilisateur et prototype

Le dashboard est développé avec Streamlit (≥ 1.32) et une couche CSS custom inspirée de la charte EFREI (Plus Jakarta Sans, palette bleu/vert/orange/rouge, design tokens type Linear/Vercel). Il est exclusivement orienté vers les besoins métier du responsable maintenance, avec un vocabulaire opérationnel (pas de F1/ROC/SHAP sur les onglets principaux).

12.1 5 onglets du dashboard

Onglet	Audience	Contenu principal
État du parc	Responsable maintenance	Vue temps réel des 20 machines, top alertes, KPIs parc
Plan d'intervention	Chef d'équipe	Table d'actions priorisées par niveau d'urgence (high/moderate/low)
Impact économique	Direction	ROI, coûts évités, comparaison avec/sans IA, économie 12k EUR seuil
Diagnostic machine	Technicien	Saisie capteurs -> prédiction temps réel -> recommandation actionnable
Détails techniques	DSI / Jury	3 sous-onglets : Données (EDA), Modèles (métriques), Interprétabilité (SHAP)

12.2 Distinction EDA vs visualisations métier

Le dashboard opère une distinction claire entre les visualisations scientifiques (matrices de confusion, courbes ROC, SHAP dot plots - reléguées dans l'onglet 'Détails techniques' pour le jury et la DSI) et les visualisations métier (jauges d'alerte, plans d'intervention, calculs d'économie - accessibles directement depuis les onglets principaux).

12.3 Lancement

```
# Depuis la racine du projet :  
streamlit run dashboard/app.py  
  
# L'application s'ouvre sur http://localhost:8501  
# Le modele est charge en lazy-load au premier appel
```

Note : les captures d'écran live du dashboard ne sont pas incluses dans ce rapport PDF car Streamlit nécessite un serveur actif pour le rendu. Le code source complet est disponible dans dashboard/app.py (~1280 lignes).

13. API REST

L'API est développée avec FastAPI 0.110, Pydantic v2 et Uvicorn. Elle expose le modèle final XGBoost via 4 endpoints REST documentés automatiquement sous /docs (Swagger UI).

13.1 Endpoints

Méthode	Route	Description
GET	/	Message d'accueil + liens vers /docs et /health
GET	/health	Status (healthy/degraded), model_loaded, api_version, timestamp_utc
GET	/model-info	model_name, métriques, features_required, operating_modes
POST	/predict	Reçoit SensorReading (9 champs), renvoie PredictionResponse

13.2 Schéma Pydantic et validation

Pydantic v2 valide automatiquement les bornes de chaque capteur (ge/le). Toute valeur hors plage retourne un code HTTP 422 avec détail de l'erreur. Exemple : vibration_rms < 0 ou operating_mode hors {normal, idle, peak} déclenchent une réponse 422 sans que le modèle soit chargé.

13.3 Exemple JSON - entrée

```
{
  "vibration_rms": 1.2,
  "temperature_motor": 48.5,
  "current_phase_avg": 12.3,
  "pressure_level": 45.0,
  "rpm": 1100,
  "operating_mode": "normal",
  "machine_type": "CNC",
  "hours_since_maintenance": 80.0,
  "ambient_temp": 22.0
}
```

13.4 Exemple JSON - sortie (cas faible risque)

```
{
  "failure_within_24h": 0,
  "probability": 0.07,
  "risk_level": "low",
  "recommendation": "Aucune action requise. Surveillance continue.",
  "model_name": "xgboost",
  "timestamp_utc": "2026-04-29T08:00:00Z"
}
```

13.5 Lancement

```
uvicorn api.main:app --reload --host 127.0.0.1 --port 8000  
# Swagger UI disponible sur http://127.0.0.1:8000/docs
```

14. Résultats et tests de démonstration

14.1 Scénarios de démonstration

Paramètre	Cas faible risque	Cas haut risque
vibration_rms	0.6 mm/s	4.5 mm/s
temperature_motor	42 degC	82 degC
rpm	1100 tr/min	2900 tr/min
operating_mode	normal	peak
machine_type	CNC	Compressor
hours_since_maintenance	30 h	520 h
Probabilité prédite	~0.07	~0.89
Risk level	low	high
Recommandation	Surveillance continue	Intervention dans les 12-24h

14.2 Suite de tests pytest

Fichier de test	Nb tests	Ce qui est vérifié
test_smoke.py	1	Imports, chemins config, schéma 15 colonnes, ColumnTransformer, MODEL_CATALOG
test_preprocessing.py	3	fit_transform produit 2D, imputation élimine 100% NaN, >= 13 feature names
test_models.py	4 modèles binaires	fit+predict_proba, multiclasse classes valides, régression floats, KeyError sur nom inval
test_evaluation.py	4	Classifieur parfait -> métriques=1.0, zeroRecall=0, to_dict clés complètes
test_api.py	3	/health 200, /predict valide 200, valeur hors plage 422

14.3 Optimisation cost-sensitive et calibration

Paramètres cost-sensitive : FN = 1 000 EUR (panne ratée), FP = 100 EUR (alerte inutile). Au seuil 0.5 : FN=30, FP=146 -> coût = 44 600 EUR. Au seuil 0.23 : FN=7, FP=256 -> coût = 32 600 EUR. Économie : 12 000 EUR par cycle de scoring.

15. Gouvernance, responsabilité et limites

15.1 Qualité et traçabilité des données

- Dataset versionné (v3.0 Kaggle CC0) avec hash SHA256 vérifié à chaque chargement via pandera (src/data_loader.py).
- RANDOM_STATE = 42 propagé dans tous les scripts via src/config.py : garantit la reproductibilité exacte des résultats.
- Modèles sérialisés en joblib avec métadonnées JSON (model_name, metrics, features_required) dans models/.
- Seuil optimal persisté dans models/optimal_threshold.json pour utilisation cohérente par le dashboard et l'API.

15.2 Limites du dataset

- Données simulées (pas un vrai flux capteurs IoT) : les distributions sont construites, pas observées en production.
- Seulement 4 types de machines (CNC, Pump, Compressor, Robotic Arm) : risque de biais si déployé sur d'autres gammes.
- Pas de dimension temporelle longue exploitable : les événements ne sont pas sériellement corrélés.

15.3 Risques et mesures de mitigation

Risque	Impact	Mitigation en place
Surapprentissage	Modèle ne généralise pas	CV 5-fold stratifiée, early_stopping MLP, test set isolé
Dérive des données	Performances dégradées en prod	Limite - non monitoré (recommandation : Evidently AI)
Biais de sélection	4 types machines uniquement	Documenté dans CONFORMITE_RNCP.md
API CORS permissive	Accès non autorisé	allow_origins=["*"] à restreindre en prod (commenté dans le code)
Dashboard sans auth	Accès local uniquement	Acceptable pour démo, OAuth2 recommandé en prod

15.4 Recommandations pour la production

- Monitoring de la dérive (data drift et concept drift) : Evidently AI ou Great Expectations.
- Réentraînement trimestriel avec injection des nouvelles données capteurs.
- Validation humaine systématique pour les cas high-risk avant intervention.
- Documentation des décisions techniques via les 4 ADR existants (docs/adr/0001-0004) et extension au fil des évolutions.

16. Limites et pistes d'amélioration

16.1 Ce qui a fonctionné

- Pipeline anti-data-leakage complet : ColumnTransformer + Pipeline sklearn garantit l'intégrité de l'évaluation.
- Couverture des 3 tâches (binaire + multiclasse + régression) sur un seul dataset : pédagogique et métier.
- 12 modèles entraînés et évalués de manière reproductible (RANDOM_STATE=42).
- Dashboard métier 5 onglets et API 4 endpoints déployés et testés.
- Interprétabilité sur 3 niveaux (native FI + permutation + SHAP).

16.2 Difficultés rencontrées

- Déséquilibre des classes (25/75) : nécessite une stratégie spécifique par modèle, SMOTE écarté, class_weight retenu.
- MLP overfitting : malgré early_stopping et régularisation L2, le MLP reste inférieur à XGBoost (F1 0.836 vs 0.886).
- Optuna documentaire : les best_params trouvés (XGBoost : n_est=500, lr=0.084, max_depth=10) ne sont pas réinjectés dans src/models.py pour ne pas invalider les résultats déjà calculés.

16.3 Pistes d'amélioration prioritaires

Priorité	Amélioration	Bénéfice attendu
1 - Haute	Réinjecter les best_params Optuna dans mode	Gain estimé +0.01-0.02 F1 sur XGBoost
2 - Haute	Monitoring drift (Evidently AI)	Détection précoce de la dégradation en production
3 - Moyenne	Déploiement cloud (Azure ML / AWS SageMa	Scalabilité, haute disponibilité, CI/CD ML
4 - Moyenne	Base PostgreSQL pour historiser les prédictio	Audit, traçabilité, ré-entraînement incrémental
5 - Basse	Authentification API (OAuth2 / JWT)	Sécurisation pour déploiement multi-utilisateurs
6 - Basse	Monitoring CO2 (CodeCarbon reports)	Écoresponsabilité documentée (RNCP C4.3)

17. Conclusion

Ce projet a transformé un dataset de capteurs industriels (24 042 observations, 15 colonnes, Kaggle CC0) en un outil décisionnel complet pour la maintenance prédictive. En suivant une démarche structurée et reproductible - de l'analyse exploratoire jusqu'au déploiement - nous avons couvert l'intégralité du cycle de vie d'un projet de Data Science supervisé.

Les chiffres clés du livrable : 12 modèles entraînés sur 3 tâches complémentaires, modèle final XGBoost binaire ($F1=0.886$, $ROC-AUC=0.995$), Random Forest pour la régression RUL ($MAE=9.57$ h, $R^2=0.673$), seuil cost-sensitive à 0.23 générant une économie estimée à 12 000 EUR par cycle, dashboard Streamlit 5 onglets orienté métier, API FastAPI 4 endpoints, et 23 tests pytest couvrant les modules critiques.

Sur le plan pédagogique, le projet couvre le Bloc 2 du RNCP40875 (Expert en Ingénierie de Données) en combinant : collecte et validation de données, preprocessing robuste (anti-leakage), modélisation supervisée multi-tâches, interprétabilité à 3 niveaux, évaluation comparative rigoureuse, et mise en production (API + dashboard). L'architecture modulaire du dépôt (13 modules src/, 10 scripts numérotés, 4 ADR, 5 fichiers de tests) garantit la maintenabilité et la défendabilité devant le jury.

Les limites identifiées - données simulées, tuning Optuna documentaire, absence de monitoring de dérive - sont documentées et accompagnées de pistes d'amélioration concrètes pour une mise en production réelle. Ce projet constitue une base solide, extensible et transférable à un contexte industriel réel.

18. Annexes

18.1 Lien GitHub

Dépôt public : <https://github.com/Adam-Blf/maintenance-predictive-industrielle>

18.2 Requirements principaux

```
numpy>=1.26
pandas>=2.1
scikit-learn>=1.4
xgboost>=2.0
joblib>=1.3
optuna>=3.5
shap>=0.44
codecarbon>=2.3
matplotlib>=3.8
seaborn>=0.13
plotly>=5.18
streamlit>=1.32
fastapi>=0.110
pydantic>=2.6
uvicorn[standard]>=0.27
fpdf2>=2.7
Pillow>=10.0
python-pptx>=0.6.21
pandera>=0.18
pytest>=8.0
pytest-cov>=4.1
```

18.3 Suite pytest - 23 tests

Fichier	Tests	Périmètre
test_smoke.py	5	Imports, schéma, config, ColumnTransformer, MODEL_CATALOG
test_preprocessing.py	3	fit_transform, imputation NaN, feature names
test_models.py	8	Binaire (4), multiclasse (2), régression (1), KeyError (1)
test_evaluation.py	4	Métriques parfaites, zero recall, to_dict
test_api.py	3	/health, /predict valide, validation 422

Exécution : `pytest tests/ -v --tb=short` (durée ~7 secondes)

18.4 Architecture Decision Records

ADR	Titre	Décision principale
0001	Stack technique	Python 3.12, scikit-learn, XGBoost, MLP sklearn (pas PyTorch/TF)
0002	Source données	Kaggle CC0 uniquement, aucun fallback synthétique
0003	Anti-data-leakage	ColumnTransformer dans sklearn.Pipeline, fit exclusivement sur train
0004	Stratégie déploiement	Streamlit local + FastAPI local, cloud en roadmap

18.5 Glossaire ML

Terme	Définition
F1-score	Moyenne harmonique de la précision et du recall - adapté aux classes déséquilibrées
ROC-AUC	Aire sous la courbe ROC (TPR vs FPR) - robuste au déséquilibre
PR-AUC	Aire sous la courbe Précision-Recall - plus informative que ROC-AUC si la classe positive est rare
Brier score	Erreur quadratique moyenne entre probabilités prédites et labels réels - mesure la calibration
SHAP	SHapley Additive exPlanations - attribue à chaque feature sa contribution marginale à la prédiction
scale_pos_weight	Hyperparamètre XGBoost = $n_{\text{négatifs}} / n_{\text{positifs}}$ - compense le déséquilibre de classes
RUL	Remaining Useful Life - durée de vie résiduelle estimée avant défaillance
ColumnTransformer	Objet scikit-learn appliquant des transformations différentes à des sous-ensembles de colonnes - clé de l'anti-leakage